



Kodro: An Offline, Self-Refining Platform for Designing Robots and Validating Their Behaviour in Realistic Simulation

Specification and Design Proposal (CA1)

Name Vaibhav Lalwani **Student ID** 201979723 **Supervisor** Keith Dures **Date** 19 June 2026

Ethics data category AO, participant category 2. The work completed to date uses simulated personas only, with no human participants and no personal data. Any subsequent study involving real users will be agreed with the supervisor and conducted under appropriate consent.

Abstract. Kodro is an offline desktop platform on which a non-expert designs a custom robot, programs it in Python, in blocks, or by voice with the support of a grounded local assistant and an automatic code reviewer, and validates its behaviour in a realistic simulated environment before committing to hardware. The robot assembled by the user parameterises the simulation, so a change to the build alters the simulated behaviour. A small language model, executed entirely on the local machine, grounds its suggestions in that specification and refines its advice from accumulated use through stored reflections and a reusable skill library, without retraining the model or accessing the network. This document states the aims and measurable objectives, the literature on which the design draws, the implementation and technology stack, the data and ethics position, and an evaluation that distinguishes usability from efficacy and from measurable self-refinement.

1. Introduction

Prototyping a physical robot is costly and slow. Component selection, wiring, and control programming demand expertise that many prospective builders lack, and control errors are expensive and unsafe to discover on real hardware. Kodro addresses this through a single offline application. The user designs a robot in a builder by selecting a chassis, a controller board such as an ESP32 or a micro:bit, and the sensors and actuators it carries; programs its behaviour in Python, in blocks, or by voice, supported by a grounded assistant and a second agent that reviews the generated code; and executes it in a simulated environment with terrain, physics, and independent moving agents such as pedestrians and traffic. The cycle of design, execution, observation, and revision is completed in simulation before any hardware is committed.

The research contribution lies in the layer above the simulator. A small language model, executed locally, assists in writing and reviewing control code and refines its behaviour through use: after each run it records the outcome, retains validated behaviours in a growing library, and draws on that record when assisting the next design. This occurs without model retraining and without network access, so the user's designs remain on the device. The project examines a focused question: the extent to which such a system can refine itself, and how dependable its validation can be, under a strict offline constraint that excludes both cloud services and any modification of model weights.

2. Aims and objectives

The aim is a functional offline platform on which a non-expert designs, programs, and validates a robot's behaviour in a realistic, agent-populated simulation prior to physical construction. Table 1 states each core objective together with the acceptance criterion against which its completion is judged.

Table 1. *Core objectives and acceptance criteria.*

Core objective	Acceptance criterion (test of done)
Custom robot design that parameterises the simulation	Changing the build measurably changes behaviour: greater mass increases battery drain, a stronger motor raises top speed, and a fitted sensor exposes its command while an absent sensor withholds it.
Sandboxed execution and scenario scoring	The robot's Python runs in a sandbox that denies file, network, and process access; a hostile program cannot reach the disk and a correct program is scored correctly.
Realistic, repeatable validation	One program executes across several randomised seeds and returns a per-run report of successes, collisions, and time to goal.
Grounded local assistant with deterministic fallback	The platform is fully usable with the network disabled and no model installed; the assistant cannot reference a component the build does not contain; the fallback engages within two seconds.
Self-refinement from use without retraining	Across a session the system reuses a stored reflection or skill, and the median number of iterations to a working design decreases.

Beyond the core artefact, broader provision for users with particular access needs, comprising a complete screen-reader path, dyslexia and low-vision modes, and a study population selected to include such users, is recorded as a future-work objective. The core artefact is specified to deliver a working and validated platform first, so the scope remains achievable within the project window.

3. Background and related work

The design proceeds from three established lines of research. The first is constructionism (Papert, 1980), the durable finding that learning is strongest when a learner builds an artefact whose behaviour can be observed and corrected. Kodro is organised around that cycle, and offers a block representation that compiles to the same Python so that the transition from blocks to text requires no change of tool.

The second concerns the conditions under which a simulator can be trusted. The principal obstacle in robotics is the gap between simulation and reality, namely the loss of performance when behaviour tuned in simulation is transferred to physical hardware. The accepted response is not a perfect simulator but domain randomisation (Tobin et al., 2017): friction, mass, sensor noise, and scene composition are varied across many runs, so that behaviour surviving the variation is likely to transfer. Kodro applies this directly, validating each design across randomised seeds and reporting the resulting distribution rather than a single run, and populating its environments with agents that avoid one another and the robot, so that a vehicle or domestic robot is evaluated among independent moving entities.

The third concerns the language model and justifies both the offline design and the careful framing of refinement. Hallucination is intrinsic to such models rather than a defect that can be removed (Huang et al., 2023), and in generated code it appears as plausible but incorrect calls (Lee et al., 2025); this is consequential when the output is control code whose errors a non-expert cannot reliably detect. The accepted mitigation is to ground generation in retrieved material rather than the model's parameters (Lewis et al., 2020); for robotics specifically, the model is constrained to compose a fixed and safe set of capabilities rather than to invent them, following the approach of programs written against a defined robot interface (Liang et al., 2023; dos Santos et al., 2026) and of grounding instructions in the robot's actual affordances (Ahn et al., 2022). Kodro grounds every suggestion in the specification the user assembled, so that the model cannot reference an unfitted sensor; grounding also removes the need for a remote service, since it operates on local material. Regarding refinement over time, weight-level self-evolution remains unsolved (Tao et al., 2024) and is therefore not claimed. Refinement is instead performed at the level of the system, through a verbal reflection loop (Shinn et al., 2023) and a growing library of validated behaviours (Wang et al., 2023), an experience-driven pattern consistent with recent self-evolving agents that improve from their own history without altering weights (Wu et al., 2025). Where multiple agents cooperate, naive configurations fail in characteristic ways (Cemri et al., 2025); a deterministic check, rather than a second model, therefore holds final authority, an automatic reviewer screens generated code beforehand following current code-review practice (Sun et al., 2025), and deterministic static analysis can identify hallucinated code directly (Khathi et al., 2026).

4. Design and implementation

The platform is implemented in Python 3.12 behind a desktop web view. Each component of the technology stack, summarised in Table 2, follows from the offline and low-resource constraint.

Table 2. *Technology stack and the rationale for each selection.*

Layer	Selection	Rationale under the offline, low-resource constraint
Language and runtime	Python 3.12, desktop web view	One installable application that retains all data on the device.
Interface	React, precompiled to a single bundle	The panelled layout is retained while the cost of a runtime compiler is paid once at build time.
Three-dimensional view	Vendored Three.js	A copy held in the project renders with the network disabled, which a content delivery network cannot guarantee.
Storage	SQLite	A single standard-library file requiring no server, easily inspected, copied, or deleted by the user.
Base model	Open-weight model, 1 to 3 billion parameters	The largest class that responds at interactive speed on a mainstream laptop without a discrete graphics card.
Domain adaptation	QLoRA fine-tuning via Unsloth and LLaMA-Factory (RTX 3050, 6 GB)	Parameter-efficient fine-tuning on the project's own robot-API corpus yields a model (kodro-coder) that uses the platform's exact command set; performed once on the development machine.
Serving	GGUF export, served by Ollama (1B drafter, 3B quality)	Ollama installs as one small application and keeps a model resident; the exported model then runs on the user's machine with no graphics card and no network.
Fallback	Deterministic rule-based path	Returns rule-based code and checks when no model is present, keeping the platform fully usable.

The designed robot is not cosmetic. Its specification is a single source of truth that the remainder of the system reads: the selected board, sensors, and motors set the mass, which scales battery drain, set the top speed, and

determine which commands exist, so that a build without an ultrasonic sensor exposes no distance reading and the grounded assistant cannot propose one. Behaviour is then validated across several randomised seeds with varied friction, mass, and sensor noise (Tobin et al., 2017). The honest limit of the model is acknowledged: fine-tuning provides domain grounding and a reliable output format rather than frontier capability, and a small local model cannot match a cloud model, so the design relies on grounding and the mandatory simulation gate rather than on model capability. The system is developed as small, independently tested increments, each merged only after a continuous integration pipeline runs the full test suite, a type checker, and a linter, so that the codebase remains releasable at every stage. Refinement is performed by three local and measurable mechanisms, summarised in Table 3, none of which modifies the model.

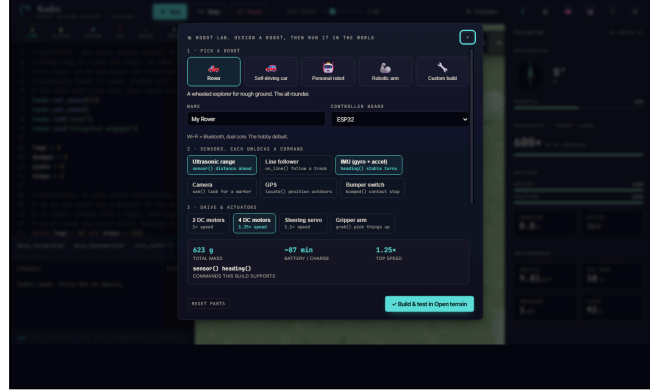


Figure 1. *The Robot Lab. The user assembles a custom robot, and the build's mass, battery life, and supported commands update as components are selected; that specification then parameterises the simulation.*

Table 3. *System-level self-refinement mechanisms.*

Mechanism	Stored record	Reuse and measure
Reflection retrieval (Shinn et al., 2023)	A short written reflection on each run's failure and success.	The most similar past reflections are retrieved into the next suggestion; reuse rate and effect on iterations are recorded.
Skill library (Wang et al., 2023)	Validated control routines as named, parameterised skills.	Skills are composed into later designs; the reuse rate is measured against a fixed floor.
Experience table	Past designs and their outcomes.	Sampled so that advice on a new build is anchored in outcomes on similar builds.

5. Data and evaluation

The project uses no external datasets and no personal data. Its content is authored by hand as structured files: a set of starter robots and a set of validation scenarios, each defined as a goal with explicit success criteria. The only data produced at runtime are the user's designs and code, the per-run results, and the system's reflections, all retained in the local file and transmitted nowhere; that record is also the material of the refinement. The local model executes from weights already present on the device. The evaluation distinguishes three questions, set out with their methods and acceptance thresholds in Table 4.

Table 4. *Evaluation questions, methods, and acceptance thresholds.*

No.	Question	Method	Primary measure and threshold
1	Usability	A panel of distinct simulated personas, each completing one session and reporting a rating and concrete defects, with named defects fixed between rounds.	Mean rating per round; an upward trend across rounds on a fixed build. Reported as a usability finding only.
2	Efficacy	Within-subject study, about sixteen non-experts, each attempting two comparable held-out scenarios with and without grounded assistance, order counterbalanced.	Task success in simulation (at least sixteen of twenty randomised runs); paired test fixed before data collection.
3	Self-refinement	Ablation across a related scenario sequence: full memory against reflection retrieval and skill library disabled.	At least a 25 percent reduction in iterations to a working design relative to the ablated arm.
4	Assistant grounding	A fixed question set including off-specification and adversarial prompts, with answers checked against the specification.	Correct at least 85 percent of the time, invention below 5 percent; grounding exceeds the same model without the specification. The 5 percent invention rate is a release gate.

The usability question has been evaluated repeatedly as the work has progressed. Table 5 records the measured trajectory. The first round was deliberately unflattering, and each subsequent round corrected the defects the personas identified before re-rating the same build, raising the mean from 5.86 to 7.36 across the rounds on the core product. The fifth round was a separate inspection of the newly added three-dimensional view, which

accounts for its lower figure. These results indicate that the tool became learnable to operate; they do not demonstrate that any user learns, which only the efficacy study can establish. The efficacy study is reported as exploratory, since sixteen participants constitute a convenience sample, so a positive result is treated as a signal to confirm at scale rather than as proof.

Table 5. Measured usability trajectory across evaluation rounds.

Round	Personas	Focus of the round	Mean (/10)
1	50	First contact with the build	5.86
2	12	After the first round of fixes	6.50
3	50	After integrating the learner model and teacher tools	7.10
4	50	After adding grounded answers and the voice route	7.36
5	8	Inspection of the new three-dimensional view	6.40

6. Ethics, project plan, and risks

No human participants have been involved to date; the personas are generated by software and are reported as such. The planned study falls under participant category 2, uses adult and older makers, provides a plain explanation, anonymises data at the point of collection, and is agreed with the supervisor under consent before it begins. One risk is specific to the technology and warrants explicit statement: the output is control code for a machine that the user may subsequently build, so a confident error could cause a physical robot to collide with a person or an obstacle. The design treats this as a safety requirement. Every suggestion is grounded in the build, the assistant declines and refers to the specification rather than guessing, and any generated program executes first within the sandboxed simulation and is checked against the scenario, so that an incorrect suggestion fails visibly on screen rather than on hardware. The mandatory simulation gate, not the model, stands between a generated program and the physical world.

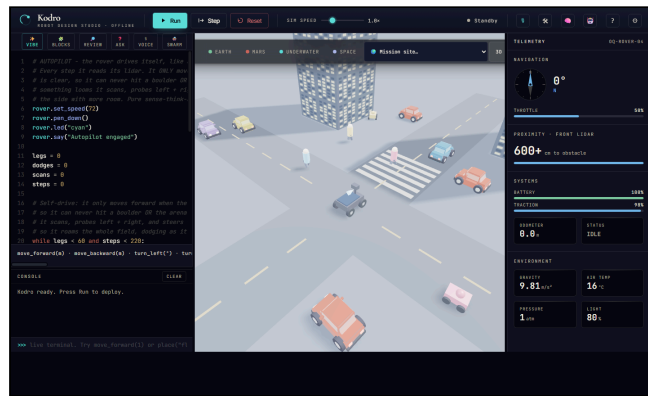


Figure 2. Validation in a realistic environment. The designed robot runs in a three-dimensional city street among traffic and pedestrians that move independently, so that its behaviour can be observed before any hardware is committed.

The plan runs from this proposal to the dissertation, incorporating the demonstration video (CA2) and the write-up (CA3), with fortnightly supervisor meetings throughout. The critical path runs through ethics approval into the design study and then into the write-up, which is why approval is sought in June with recruitment lead time; the write-up begins during the study on the parts that do not depend on its results, with a buffer before the deadline. The principal risks and their contingencies are given in Table 6. The project is intended to satisfy the six BCS honours outcomes: it engages current problems (hallucination and grounding, honest offline refinement, and the simulation-to-reality gap), justifies its techniques against the literature, and is reported together with its weak early results and the corrections that followed rather than only its strongest figures. The originality is not a new algorithm; retrieval grounding, a sandboxed gate, an experience memory, a skill library, and domain randomisation are each established, and the offline constraint limits how far any one can be pressed. The contribution is the demonstration that these components combine into an honest design-and-validation platform that runs entirely on one machine, and a measured account of how far offline self-refinement extends when modification of weights is excluded. The constraint defines the contribution rather than diminishing it.

Table 6. *Principal risks and contingencies.*

Risk	Contingency	Likelihood	Impact
Generated control code is unsafe	Every program executes first in the sandboxed simulation against the scenario; grounding and the reviewer identify invented commands beforehand.	Medium	High
Local model too weak for some suggestions	Grounding ties suggestions to the build and the model declines rather than guesses; a stronger local model may be configured.	Medium	Medium
Ethics approval or recruitment delayed	Approval is sought in June with lead time; if it slips, the participant-free evaluations proceed and the write-up is not blocked.	Medium	Medium
Schedule overrun	The build is releasable at every stage, so an earlier version remains a valid deliverable.	Medium	High

References

- Ahn, M., Brohan, A., Brown, N. et al. (2022) Do as I can, not as I say: grounding language in robotic affordances. *arXiv:2204.01691*.
- Cemri, M., Pan, M.Z., Yang, S. et al. (2025) Why do multi-agent LLM systems fail? *arXiv:2503.13657*.
- dos Santos, V.G., Khadraoui, I., Farhat, I. et al. (2026) ALRM: agentic LLM for robotic manipulation. *arXiv:2601.19510*.
- Huang, L., Yu, W., Ma, W. et al. (2023) A survey on hallucination in large language models. *ACM Transactions on Information Systems* (arXiv:2311.05232).
- Khati, D., Rodriguez-Cardenas, D., Pantzer, P. and Poshyvanyk, D. (2026) Detecting and correcting hallucinations in LLM-generated code via deterministic AST analysis. *arXiv:2601.19106*.
- Lee, Y., Song, J.Y., Kim, D. et al. (2025) Hallucination by code generation LLMs: taxonomy, benchmarks, mitigation, and challenges. *arXiv:2504.20799*.
- Lewis, P., Perez, E., Piktus, A. et al. (2020) Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33, pp. 9459 to 9474.
- Liang, J., Huang, W., Xia, F. et al. (2023) Code as policies: language model programs for embodied control. *IEEE International Conference on Robotics and Automation (ICRA)*.
- Papert, S. (1980) *Mindstorms: children, computers, and powerful ideas*. New York: Basic Books.
- Reza, Z., Mazur, A., Dugdale, M.T. and Ray-Chaudhuri, R. (2025) Small models, big support: a local LLM framework for educator-centric content creation and assessment with RAG and CAG. *arXiv:2506.05925*.
- Shinn, N., Cassano, F., Berman, E. et al. (2023) Reflexion: language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems*, 36 (arXiv:2303.11366).
- Sun, T., Xu, J., Li, Y. et al. (2025) BitsAI-CR: automated code review via LLM in practice. *arXiv:2501.15134*.
- Tao, Z., Lin, T.E., Chen, X. et al. (2024) A survey on self-evolution of large language models. *arXiv:2404.14387*.
- Tobin, J., Fong, R., Ray, A. et al. (2017) Domain randomization for transferring deep neural networks from simulation to the real world. *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 23 to 30.
- Wang, G., Xie, Y., Jiang, Y. et al. (2023) Voyager: an open-ended embodied agent with large language models. *arXiv:2305.16291*.
- Wang, W., Li, Y., Jiao, L. and Yuan, J. (2026) Ro-SLM: onboard small language models for robot task planning and operation code generation. *arXiv:2604.10929*.
- Wu, R., Wang, X., Mei, J. et al. (2025) EvolveR: self-evolving LLM agents through an experience-driven lifecycle. *arXiv:2510.16079*.

Generative artificial intelligence assisted the development of the software and the drafting of this document, acknowledged in line with University guidance. All technical claims describe the working system, the evaluation figures are reported as measured, and the references were verified at the time of writing.